# Report on

**“Tutorials”**

For

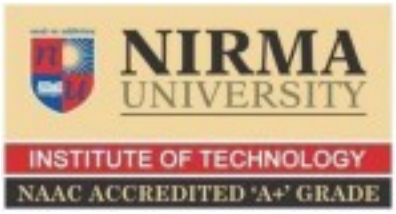
**Software Project Management - 4CS504ME25 B.**

**Tech. Semester IV**

## Prepared By :

Gandhi Urva Yogeshkumar (23BCE078)

Gajnotar Rakshit (23BCE077)




**Institute of Technology,**
**Nirma University**
**Ahmedabad - 382481**

**October 2026**

## Aim

To analyze maintenance requests, classify maintenance types, estimate maintenance effort and prepare a maintenance plan for the Hospital Management System (HMS), including a comparison of project management processes for fresh versus maintenance projects.

## Objective

Classify post-deployment maintenance requests into Corrective, Adaptive, Perfective and Preventive categories according to ISO/IEC 14764 standards.

Analyze ten realistic maintenance requests originating from clinical staff and hospital administrators during the PulseCare HMS operational pilot.

Estimate maintenance effort, duration and staffing utilizing the Annual Change Traffic (ACT) model and Belady-Lehman equations.

Establish a formal Software Maintenance Plan covering Change Request (CR) lifecycles, Change Control Board (CCB) governance, Service Level Agreements (SLAs) and release scheduling.

Conduct a detailed comparative analysis contrasting project managment processes, lifecycle constraints, estimation uncertainty and risk profiles between Fresh (New) projects and Maintenance projects.

Define regression testing and release procedures to prevent defect leakage into operational hospital workflows.

## 1. Introduction

Software maintenance is the totality of activities required to provide cost-effective support to a software system after deployment. In healthcare information systems, maintenance typically accounts for 65% to 80% of total lifecycle software costs. Once a hospital management system goes live, it must operate continuously around the clock. Clinical staff rely on the system for emergency registrations, outpatient consultations, diagnostic lab orders and discharge billing.

PulseCare HMS completed initial release candidate verification at Week 16 and entered a 60-day operational pilot trial across a 150-bed community health center. During this pilot period, operational feedback, clinical edge cases and infrastructure updates generated change requests. Managing these changes without introducing software regressions requires a structured maintenance framework led by the project team: Gandhi Urva (Backend Lead, 23BCE078) and Gajnotar Rakshit (Frontend Lead, 23BCE077).

## 2. Maintenance Request Analysis and Classification

Software maintenance is categorized into four standard types defined by ISO/IEC 14764 and IEEE standards:

1. Corrective Maintenance: Reactive modification of software to correct faults and runtime errors discovered during operational use.

2. Adaptive Maintenance: Modification performed to keep software usable in a changed or changing operating environment (such as operating system updates, external API changes, legal mandates and library upgrades).

3. Perfective Maintenance: Modification performed to enhance software performence, improve user experience or add new user-requested capabilities.

4. Preventive Maintenance: Modification performed to detect and correct latent faults before they manifest as operational failures, refactor complex code and reduce technical debt.

During the operational pilot, ten concrete Change Requests (CR-01 through CR-10) were logged, evaluated and classified as detailed below:

| CR ID | Module | Logged By | Problem / Request Description | Technical Work Required | Category | Priority |
|---|---|---|---|---|---|---|
| CR-01 | Pharmacy | Chief Pharmacist | Concurrent dispenses of the same fast-moving medicine allow stock levels to drop below zero. | Implement MongoDB atomic conditional update ($inc with $gte check) to enforce inventory floor. | Corrective | Critical (P1) |
| CR-02 | Billing | Hospital Accountant | Sub-cent rounding discrepancies appear on multi-item pharmacy invoices with mixed GST tax brackets (5%, 12%, 18%). | Refactor calculateInvoice Totals() to apply rounding per line item before subtotal aggregation. | Corrective | High (P2) |
| CR-03 | Patient | Medical Records Officer | National Health Authority mandated Ayushman Bharat Digital Mission (ABDM) M1/M2 health ID generation compliance. | Integrate ABDM sandbox REST APIs for Aadhaar OTP authentication and ABHA address linking. | Adaptive | High (P2) |
| CR-04 | Backend Core | DevOps / Security Admin | Node.js LTS security bulletin flagged prototype pollution vulnerabilities in third-party utility packages. | Upgrade Express, Mongoose and Zod dependencies to latest patch versions and verify test suites. | Adaptive | Medium (P3) |
| CR-05 | Appointments | OPD Receptionist | Patients arrive late or miss appointments due to lack of automated appointment reminders. | Integrate Twilio and Gupshup SMS/WhatsApp gateway to trigger automated reminder 2 hours prior to slot. | Perfective | Medium (P3) |
| CR-06 | Frontend Portal | Patient User Group | Patients cannot easily book recurring follow-up visits or switch between family dependent accounts on mobile. | Redesign patient dashboard with one-click follow-up booking and family profile switcher. | Perfective | Medium (P3) |

| CR ID | Module | Logged By | Problem / Request Description | Technical Work Required | Category | Priority |
|---|---|---|---|---|---|---|
| CR-07 | Admin Billing | Finance Director | Hospital auditors require daily cash, card and insurance revenue reconciliation reports exported to Excel and PDF. | Develop server-side report generator generating formatted Excel (.xlsx) and PDF reconciliation sheets. | Perfective | Low (P4) |
| CR-08 | Appointments | Backend Developer | Appointment router has grown to 650 lines of code; slot availability queries slow down as appointment records grow. | Split monolithic router into modular sub-routes; create MongoDB compound index on { doctorId: 1, date: 1 }. | Preventive | Medium (P3) |
| CR-09 | Backend Cache | System Architect | Frequent queries for doctor OPD weekly schedules cause repetitive database reads on static department data. | Deploy Redis caching layer for doctor schedule lookups with 15-minute TTL invalidation on updates. | Preventive | Medium (P3) |
| CR-10 | Patient | OPD Clerk | Search bar crashes with unhandled exception when users paste phone numbers with leading plus signs or hyphens. | Sanitize search input by stripping non-numeric symbols before regex evaluation in search query. | Corrective | High (P2) |

## 2.1 Maintenance Distribution Analysis

The distribution of our ten maintenance requests was compared against the empirical Lientz-Swanson maintenance distribution benchmark:

| Maintenance Type | Count in PulseCare HMS | Sample Percentage | Lientz-Swanson Benchmark | Analytical Observation |
|---|---|---|---|---|
| Corrective Maintenance | 3 Requests | 30.0% | 20% - 25% | Slightly elevated during pilot as clinical edge cases and concurrency issues surfaced. |
| Adaptive Maintenance | 2 Requests | 20.0% | 18% - 25% | Driven by national healthcare compliance mandates (ABDM) and Node.js dependency updates. |

| Maintenance Type | Count in PulseCare HMS | Sample Percentage | Lientz-Swanson Benchmark | Analytical Observation |
|---|---|---|---|---|
| Perfective Maintenance | 3 Requests | 30.0% | 50% - 60% | Focuses on user convenience features (SMS reminders, dashboard shortcuts, revenue export). |
| Preventive Maintenance | 2 Requests | 20.0% | 5% - 10% | Proactively addresses code refactoring, query indexing and Redis caching to prevent debt. |
| Total | 10 Requests | 100.0% | 100.0% | Balanced maintenance portfolio addressing immediate stability and scalability. |

## 3. Maintenance Effort and Cost Estimation

To budget engineering effort for software maintenance, we applied the empirical Annual Change Traffic (ACT) model combined with Belady-Lehman effort equations.

### 3.1 Annual Change Traffic (ACT) Model

Software size of PulseCare HMS baseline: 28,500 Lines of Code (28.5 KLOC). Based on pilot change logs, our projected annual change traffic is estimated as:

- Estimated Lines Added (KLOC_add): 2,100 LOC (ABDM integration, SMS gateway, revenue reporting).
- Estimated Lines Modified (KLOC_mod): 2,600 LOC (concurrency fixes, tax rounding, dashboard redesign).
- Estimated Lines Deleted (KLOC_del): 850 LOC (obsolete prototype scripts and legacy routing code).

ACT = ( Added + Modified + Deleted ) / Total Baseline LOC
ACT = ( 2,100 + 2,600 + 850 ) / 28,500 = 5,550 / 28,500 = 0.1947 (19.47% annual traffic)

### 3.2 Annual Maintenance Effort (AME) Calculation

Annual Maintenance Effort is computed using the standard formula:

AME = ACT * PDE * PAF

Where:
- ACT = 0.1947 (Annual Change Traffic)
- PDE = Project Development Effort = 8.86 Person-Months (from T10 tracking baseline)
- PAF = Maintenance Process Adjustment Factor = 1.12 (reflects strict TypeScript typing, modular architechture and automated regression tests that lower maintenance friction)

AME = 0.1947 * 8.86 * 1.12 = 1.93 Person-Months
Converting to person-hours (at 160 hours per person-month): 1.93 * 160 = 308.8 Person-Hours (~310 hours annually, or ~26 hours monthly).

### 3.3 Effort Breakdown per Maintenance Request

The total maintenance effort was allocated across the ten logged change requests:

| CR ID | Short Title | Type | Story Points | Estimated Hours | Assigned Lead | Target Window |
|---|---|---|---|---|---|---|
| CR-01 | Atomic stock decrement | Corrective | 3 SP | 16 Hours | Urva Gandhi | Sprint M+1 (Day 1-2) |
| CR-02 | GST invoice rounding fix | Corrective | 3 SP | 16 Hours | Urva Gandhi | Sprint M+1 (Day 3-4) |
| CR-03 | ABDM M1/M2 health ID API | Adaptive | 8 SP | 56 Hours | Urva Gandhi | Sprint M+2 |
| CR-04 | Node.js dependency upgrade | Adaptive | 2 SP | 12 Hours | Urva Gandhi | Sprint M+1 |
| CR-05 | Automated SMS/WhatsApp notifications | Perfective | 5 SP | 36 Hours | Rakshit Gajnotar | Sprint M+2 |
| CR-06 | Patient dashboard redesign | Perfective | 5 SP | 40 Hours | Rakshit Gajnotar | Sprint M+3 |
| CR-07 | Reconciliation revenue export | Perfective | 5 SP | 32 Hours | Rakshit Gajnotar | Sprint M+3 |
| CR-08 | Appointment router split and indexing | Preventive | 3 SP | 20 Hours | Urva Gandhi | Sprint M+2 |
| CR-09 | Redis schedule caching layer | Preventive | 5 SP | 36 Hours | Urva Gandhi | Sprint M+3 |
| CR-10 | Search regex input sanitization | Corrective | 2 SP | 10 Hours | Rakshit Gajnotar | Sprint M+1 |
| Total | Consolidated Maintenance Work | -- | 41 SP | 274 Hours | Both Leads | Quarterly Cycle |

## 4. Software Maintenance Plan and Governance

Our Software Maintenance Plan defines change management procedures, decision authorities and release cadences to preserve system stability.

### 4.1 Change Request (CR) Lifecycle

Every maintenance change follows an eight-stage lifecycle:

1. CR Submission: User or engineer submits a structured Change Request detailing observed behavior, expected behavior and affected clinical workflow.
2. Technical Triage: Quality Lead validates the request, verifies reproduction steps and assigns initial severity.
3. Impact Analysis: Architect evaluates codebase dependencies, regression risks and estimated person-hours.
4. CCB Review: Change Control Board evaluates operational benefits versus risk and approves, defers or rejects the request.
5. Implementation: Engineer implements the change in an isolated bugfix/ or feature/ branch.
6. Regression Testing: Automated Jest suites (340 tests) execute to confirm zero regression; new test cases added for the fix.
7. Staged Deployment: Verified artifact merged and deployed via hotfix or scheduled release.

8. Verification & Sign-off: User confirms resolution and Change Request is officially closed in the tracking register.

### 4.2 Change Control Board (CCB) Governance

The Change Control Board governs all production modifications. The board comprises:

- Lead Software Architect: Gandhi Urva (Backend Lead) - evaluates database integrity, API backwards-compatability and security.
- Frontend Systems Lead: Gajnotar Rakshit (Frontend Lead) - evaluates UI usability, state managment and client compatability.
- Hospital Stakeholder Lead: Clinical Operations Representative - represents doctor and nurse workflow priorities.
- Quality Assurance Lead: Academic Mentor / Project Guide - reviews architectural modifications and milestone compliance.

### 4.3 Maintenance Service Level Agreements (SLAs)

| SLA Priority Tier | Incident Criteria | Response SLA | Workaround SLA | Permanent Resolution SLA |
|---|---|---|---|---|
| Tier 1 (Critical) | System outage, database corruption, patient identity mismatch, billing calculation freeze. | < 1 Hour | < 4 Hours | < 24 Hours (Emergency Hotfix release) |
| Tier 2 (High) | Core module impaired (e.g. appointment booking broken for a doctor department); no workaround. | < 2 Hours | < 8 Hours | < 48 Hours (Point patch release) |
| Tier 3 (Medium) | Non-blocking functional defect, minor regulatory update, dependency advisory, UI glitch. | < 8 Hours | < 24 Hours | < 14 Days (Next sprint release) |
| Tier 4 (Low) | Cosmetic layout enhancement, optional report format, minor refactoring. | < 24 Hours | N/A | < 30 Days (Scheduled quarterly release) |

## 5. Follow-Up Comparative Analysis: Fresh Projects vs Maintenance Projects

Software project managment differs substantially between fresh green-field implementation and ongoing brown-field maintenance. While green-field projects focus on architectural synthesis and rapid feature creation, maintenance projects operate within rigid architectural constraints where the primary objective is defect containment and regression prevention.

The table below contrasts project managment processes across twelve structural dimensions:

| Project Management Dimension | Fresh / New (Green-field) Projects | Maintenance / Evolution (Brown-field) Projects |
|---|---|---|
| 1. Project Scope & Charter | Broad, expansive and defined from scratch. Scope is structured by high-level epics, core functional modules and overarching vision. | Granular, constrained and incremental. Scope is defined by specific Change Requests (CRs), defect tickets and regulatory patches. |

| Project Management Dimension | Fresh / New (Green-field) Projects | Maintenance / Evolution (Brown-field) Projects |
|---|---|---|
| 2. Architechture & Design Freedom | High architectural freedom. Developers select frameworks, databases, library ecosystems and folder structures freely. | Low design freedom. Engineers must conform to established architectures, legacy data schemas and existing API contracts. |
| 3. Requirements Volatility | Requirements emerge through customer interviews, prototypes and competitive analysis; high ambiguity in early sprints. | Requirements are tightly scoped around existing system behavior, reproduction steps and concrete business modifications. |
| 4. Estimation Techniques | Sizing via Function Points, Use Case Points and COCOMO II; higher estimation variance (15% to 30% uncertainty). | Sizing via Annual Change Traffic (ACT), Story Point deltas and historical ticket closure rates; lower variance (5% to 10%). |
| 5. Schedulling & Milestones | Organized around sequential engineering milestones (M1 Inception, M2 Schema Freeze, M5 Integration, M6 Handover). | Organized around continuous service levels, sprint maintenance windows and emergency hotfix release cycles. |
| 6. Testing Focus & Strategy | Focus on green-field unit testing, API test creation and initial end-to-end user journey validation. | Heavy emphasis on regression testing to verify that bug fixes or enhancements do not break previously functioning modules. |
| 7. Risk Management Profile | Dominant risks: architectural failure, schedule slippage, scope creep, technology unsuitability and team burnout. | Dominant risks: regression bugs, ripple effects, unintended database corruption, obsolete libraries and knowledge erosion. |
| 8. Team Staffing & Cognitive Load | Requires system architects, UI designers and full-stack builders; high initial creative energy and collaborative design. | Requires diagnostic troubleshooting skills, reverse-engineering capability, patience and rigorous testing discipline. |
| 9. Configuration Management | Feature branching from develop branch; focus on establishing the initial repository structure and merge policies. | Strict release tagging, dual hotfix and maintenance branching, backporting fixes across supported customer versions. |
| 10. Stakeholder Interaction | Interactive visioning sessions, sprint review demos and requirement elicitation workshops with client sponsors. | Ongoing operational support, helpdesk ticket triage, service review meetings and SLA uptime reporting. |
| 11. Documentation Dependence | Authors initial SRS, Software Architecture Document, ER diagrams and API contracts for downstream implementation. | Heavily relies on existing documentation to understand system behavior; failure occurs if legacy code lacks documentation. |
| 12. Success Metrics & KPIs | Velocity, milestone on-time delivery rate, Schedule Performance Index (SPI) and Cost Performance Index (CPI). | Mean Time to Repair (MTTR), SLA compliance rate, Defect Re-open Rate, regression rate and system availability uptime. |

In PulseCare HMS, the transition from fresh development to maintenance shifted our focus from rapid feature coding to stability preservation. While developing Sprint 1 through 6 required building 28,500 lines of code, maintaining the application demands rigorous regression verification to ensure that modifications to billing tax rates (CR-02) or stock concurrency (CR-01) do not break doctor scheduling or patient record access.

## 6. Conclusion

Software maintenance is an essential phase of the software engineering lifecycle that protects software longevity and business value. By analyzing ten real-world change requests from the hospital pilot and classifying them into standard categories, our team established an empirical maintenance plan. The Annual Change Traffic model estimated an annual maintenance effort of 1.93 Person-Months (~310 hours), providing realistic staffing expectations for Urva and Rakshit. Furthermore, the comparative analysis between fresh and maintenance projects

highlighted that while initial development values rapid feature assembly, successful maintenance demands strict regression testing, disciplined change board control and predictable SLA adherence.